Metro Ticket Booking System using ServiceNow

Project Report, Design, Requirements & Implementation Guide

Prepared: 15 October 2026 | Prototype / Academic Project

Source material used

Prepared using the uploaded Ideation Phase and Requirement Analysis template archives. The three additional Drive folders linked by the user were not included in the uploaded ZIP files at the time of generation.

1. Executive Summary

The Metro Ticket Booking System digitizes metro ticket booking through a ServiceNow-based request workflow. A passenger selects origin and destination stations, travel date, and passenger count; the system validates the journey, calculates an indicative fare, creates a ticket record, generates a QR payload, and sends a confirmation notification.

This package is a project prototype and implementation guide. It is not connected to a live metro operator, payment gateway, station database, or fare authority. Before operational use, fare rules, station data, ticket validation, payment processing, privacy, and security must be approved by the relevant operator.

2. Problem Statement

Passengers may need to queue at ticket counters or use separate applications to plan and obtain a metro ticket. Manual ticket issuance can create delays, increase staff workload, and make ticket information harder to retrieve. A simple digital booking workflow can reduce friction and provide an immediately accessible ticket confirmation.

3. Objectives

Provide a guided ticket booking request using Service Catalog.

Capture origin, destination, travel date, and passenger count with validation.

Calculate a prototype fare automatically using configurable sample rules.

Create a durable ticket record with a unique ticket number and QR payload.

Use Flow Designer to orchestrate validation, ticket creation, and notifications.

Send booking confirmation through ServiceNow notifications.

Support accessibility, auditability, and maintainable station/fare configuration.

4. Scope and Assumptions

In scope: passenger booking request, basic route validation, fare estimate, ticket record, QR payload, booking confirmation, and administrator-maintained station/fare configuration.

Out of scope for this prototype: live transit APIs, actual payment collection/refunds, gate hardware integration, anti-fraud verification, real-time service disruptions, and official ticket validity.

Sample fare rule used by the demo: base fare INR 10 plus INR 5 per station interval, multiplied by passenger count. Replace this with the operator's approved fare matrix before real use.

Ticket booking and QR generation in the included local Python demo are independent of ServiceNow. The ServiceNow scripts are illustrative and must be adapted and tested in a sub-production instance.

5. Users and Roles

Passenger: submits booking requests and receives ticket details.

Metro support agent: reviews failed or exceptional bookings and assists passengers.

Metro administrator: manages station data, fare configuration, catalog variables, access controls, and notification templates.

Ticket inspector/gate validator: scans a QR code and verifies ticket status against a trusted server-side record. A QR code by itself is not proof of validity.

6. Functional Requirements

FR-01: Passenger can submit origin, destination, travel date, and passenger count.

FR-02: Origin and destination must be different valid stations.

FR-03: Travel date must be valid and passenger count must be within configured limits.

FR-04: System calculates fare using configurable fare rules and shows the estimate before submission where supported.

FR-05: System creates a booking/ticket record with a unique identifier and status.

FR-06: System generates a QR payload that references the ticket identifier; sensitive personal data must not be embedded.

FR-07: System sends a confirmation notification containing ticket number, route, travel date, fare, and a link or attachment to the ticket.

FR-08: Authorized staff can view booking details and update ticket status.

FR-09: System records important changes for audit and troubleshooting.

7. Non-Functional Requirements

Usability: clear labels, accessible keyboard navigation, concise errors, and mobile-friendly forms.

Security: least-privilege ACLs, server-side validation, authenticated ticket verification, and no secrets in client scripts.

Reliability: graceful error handling, unique ticket identifiers, and controlled retries for notifications.

Performance: target a responsive booking experience under normal load; validate with instance-specific performance tests.

Availability: follow the availability and recovery commitments of the hosting ServiceNow instance.

Scalability: use platform-native records and flows, avoid unnecessary synchronous external calls, and monitor transaction volumes.

Privacy: collect only necessary passenger information, apply retention rules, and follow the operator's privacy requirements.

8. Proposed Architecture

Passenger → Service Catalog item / Employee Center or Service Portal → Catalog variables and validation → Flow Designer → server-side fare/ticket logic → booking record and ticket record → QR payload / optional QR image service → ServiceNow Notification → passenger email or portal ticket view.

Supporting records: station reference data, fare configuration, ticket bookings, and audit history. A production deployment should expose a secure verification endpoint for scanners; the endpoint must check ticket status, expiry, and reuse policy server-side.

9. ServiceNow Data Model

Prototype booking table: x_metro_booking. Suggested fields: number (auto-number), requested_for (reference to User), origin_station (string or reference), destination_station (string or reference), travel_date (date), passenger_count (integer), fare_amount (currency/decimal), ticket_status (choice: Requested, Confirmed, Cancelled, Used, Failed), ticket_id (unique string), qr_payload (string), notification_status (choice), and booking_source (choice).

Station table: x_metro_station with station_code, station_name, line, sequence or route metadata, active flag. Fare configuration table: x_metro_fare_rule with base_fare, per_station_fare, minimum_fare, effective_from, effective_to, and active flag. Restrict writes to authorized administrators.

10. Service Catalog and Flow Designer

Create a catalog item named 'Book a Metro Ticket'. Variables: origin_station, destination_station, travel_date, passenger_count, and optional contact preference. Use reference/choice variables backed by active station records where possible.

Catalog client scripts may improve usability, but server-side validation is authoritative. Validate that both stations exist and are active, stations differ, date is permitted, and passenger count is within policy.

Flow Designer trigger: Service Catalog → Requested Item created. Steps: read variables; validate input; calculate fare; create booking/ticket record; generate ticket ID and QR payload; update RITM/booking status; send confirmation notification; log errors and route failures to support.

11. Fare Calculation

Prototype formula: station_intervals = absolute difference between station sequence values; fare_per_passenger = max(minimum_fare, base_fare + per_station_fare × station_intervals); total_fare = fare_per_passenger × passenger_count.

Do not assume station sequence difference represents a valid route across multiple lines. Production fare calculation should use an approved fare matrix or route engine, including transfers, fare caps, concessions, and special zones as applicable.

12. QR Code and Ticket Validation

The QR code should contain an opaque ticket ID or a signed, short-lived verification URL, not names, email addresses, payment data, or other unnecessary personal data. The inspector app should call a trusted verification service and check that the ticket exists, is confirmed, is within its validity window, and has not already been used if single-use policy applies.

The included QR generator is a local demonstration. Its QR image encodes a demo ticket payload and does not make the ticket a valid metro ticket.

13. Security, Accessibility, and Compliance

Use role-based access control and table/field ACLs. Never trust a client-provided fare; recompute fare on the server. Validate all input, escape output, and use secure APIs. Keep credentials in platform credential records, not source code.

Provide meaningful field labels, keyboard access, visible focus, sufficient contrast, and error messages that explain how to recover. Test with screen readers and mobile devices.

Before launch, obtain operator approval for fare policy, ticket validity, payment, refunds, data retention, accessibility, and applicable transport/security requirements.

14. Testing Plan

Happy path: valid origin/destination/date/passenger count creates one booking, calculates expected fare, generates a ticket ID/QR, and sends notification.

Validation: same origin and destination; inactive station; invalid date; zero, negative, or excessive passenger count; missing required fields.

Security: unauthorized table read/write, tampered fare input, forged ticket ID, expired ticket, cancelled ticket, repeated scan, and direct URL access.

Resilience: notification failure, duplicate flow trigger, record creation failure, and retry behavior.

Accessibility and usability: keyboard-only booking, focus order, labels, readable errors, zoom, mobile viewport, and screen-reader labels.

15. Implementation Roadmap

Sprint 1: confirm operator rules and stations; define data model and access controls.

Sprint 2: configure catalog item, variables, UI policies, and server-side validation.

Sprint 3: implement fare logic, booking records, and Flow Designer orchestration.

Sprint 4: configure notifications, QR ticket view, and staff support views.

Sprint 5: test security, accessibility, failure paths, and acceptance criteria.

Sprint 6: pilot with sample data, collect feedback, and prepare production approval.

16. Acceptance Criteria

A valid booking request creates one ticket record with a unique ticket number.

The fare is calculated server-side using the configured fare rule.

A QR image can be generated from the ticket identifier/payload.

A confirmation notification is sent or its failure is recorded for retry.

Invalid or unauthorized requests are rejected without issuing a confirmed ticket.

Authorized staff can locate the booking and verify status.

17. Conclusion

The proposed ServiceNow workflow provides a practical foundation for digital metro ticket booking by combining a guided catalog request, Flow Designer automation, server-side fare logic, ticket records, QR payloads, and notifications. It remains a prototype until connected to approved station/fare data and a secure ticket validation service.

Appendix A — Sample station data

Station Code	Station Name	Sequence (demo)
ST01	Central	1
ST02	Museum	2
ST03	City Park	3
ST04	University	4
ST05	Airport Road	5